

CCM101 — Module 4 Lab Activities

Containerization and Cloud-Native Computing Tools: Ubuntu Server + Docker

Submit: PDF with screenshots + short answers File Name:

CCM101_Mod4_LastName_FirstName.pdf

LAB 4.1 — Install Docker & Test It

Goal: Install Docker and confirm it works.

Steps

1. Update your server:

```
sudo apt update && sudo apt upgrade -y
```

2. Install Docker and Docker Compose:

```
sudo apt install docker.io docker-compose -y
```

3. Run this so you don't need sudo every time:

```
sudo usermod -aG docker $USER
```

4. Check versions:

```
docker --version  
docker compose version
```

5. Run your first container:

```
docker run hello-world
```

Screenshot

```
PS E:\dev-1tb\CCM> docker --version
Docker version 29.7.2, build a7dcaa6
PS E:\dev-1tb\CCM> docker-compose --version
Docker Compose version v5.5.1
PS E:\dev-1tb\CCM> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Written Answer (Lab 4.1)

Question: Why add your user to the docker group?

Answer: Adding your user to the docker group allows you to run Docker commands without needing to prefix them with `sudo` every time. By default, the Docker daemon binds to a Unix socket owned by the root user, so only root or users in the docker group can communicate with it. Adding your user to this group grants the necessary permissions to interact with the Docker daemon directly, making it more convenient and streamlining your workflow when managing containers, images, and other Docker resources.

LAB 4.2 — Work with Images & Containers

Goal: Download images, start/stop containers, and see how they work.

Steps

1. Download 3 images:

```
docker pull nginx
docker pull alpine
docker pull ubuntu
```

Pull nginx

```
PS E:\dev-1tb\CCM> docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
f340c1b7c1d6: Pull complete
a44b5c8be616: Pull complete
02fc02c4ab8d: Pull complete
956faab5efb3: Pull complete
6310eb16bf42: Pull complete
07db7bf2649b: Pull complete
c12f394dea35: Pull complete
25202a7045eb: Download complete
76f27c02d218: Download complete
```

Pull alpine

```
PS E:\dev-1tb\CCM> docker pull alpine
Using default tag: latest
latest: Pulling from library/alpine
55afa1ecc21d: Pull complete
56dceff11b33: Download complete
f5124fb579e2: Download complete
```

Pull ubuntu

```
PS E:\dev-1tb\CCM> docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
6a7c4f6d8c38: Pull complete
08f5f5b2a2b0: Pull complete
c04a683f373c: Download complete
```

2. See your images:

```
docker images
```

```
PS E:\dev-1tb\CCM> docker images
```

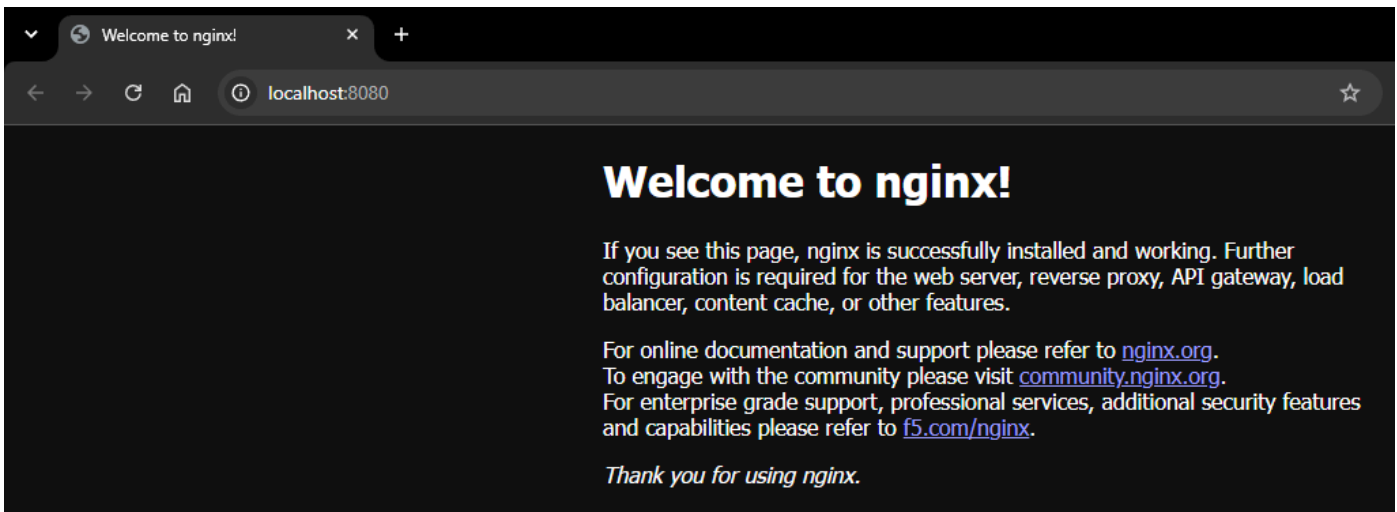
IMAGE	ID	DISK USAGE	CONTENT SIZE
alpine:latest	28bd5fe8b56d	13MB	3.93MB
hello-world:latest	5e2309035332	25.9kB	9.49kB
nginx:latest	05b8cb60c354	253MB	69.2MB
ubuntu:latest	513c074113a8	161MB	45.3MB

3. Run a container and name it:

```
docker run -d --name my-web -p 8080:80 nginx
```

```
PS E:\dev-1tb\CCM> docker run -it --name my-ubuntu ubuntu
root@e26b6b34ca11:/# bomb
bash: bomb: command not found
root@e26b6b34ca11:/# echo "My name is bryx" > myfile.txt
root@e26b6b34ca11:/# exit
exit
PS E:\dev-1tb\CCM> docker start -ai my-ubuntu
root@e26b6b34ca11:/# cat myfile.txt
My name is bryx
root@e26b6b34ca11:/#
```

4. Open browser to <http://localhost:8080>:



5. Cleanup:

```
docker stop my-web
docker rm my-web
docker rmi nginx alpine ubuntu
```

```
PS E:\dev-1tb\CCM> docker stop my-ubuntu my-web
my-ubuntu
my-web
PS E:\dev-1tb\CCM> docker rm my-ubuntu my-web
my-ubuntu
my-web
PS E:\dev-1tb\CCM> docker rmi alpine
Untagged: alpine:latest
Deleted: sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db86916198a6ee
PS E:\dev-1tb\CCM> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
PS E:\dev-1tb\CCM> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	5e2309035332	25.9kB	9.49kB	
nginx:latest	05b8cb60c354	253MB	69.2MB	
ubuntu:latest	513c074113a8	161MB	45.3MB	

```
PS E:\dev-1tb\CCM>
```

Written Answer (Lab 4.2)

Question: What is the difference between an image and a container?

Answer: A Docker image is a read-only template that contains a set of instructions for creating a container. It includes the application code, runtime, libraries, environment variables, and configuration files needed to run the application. A container, on the other hand, is a running instance of an image. It is a lightweight, isolated environment that executes the application defined by the image. You can create multiple containers from the same image, and each container runs independently with its own writable layer on top of the read-only image layers.

LAB 4.3 — Keep Data Safe (Volumes & Bind Mounts)

Goal: Learn how to persist container data using Named Volumes and Bind Mounts.

Part A — Named Volume

```
docker volume create my-vol
docker run -d --name vol-test -v my-vol:/app nginx
docker exec -it vol-test sh -c "echo 'Data in volume' > /app/data.txt"
docker stop vol-test
docker rm vol-test
docker run -it --rm -v my-vol:/app alpine cat /app/data.txt
```

Screenshot

```

PS E:\dev-1tb\CCM> docker volume create my-vol
my-vol
PS E:\dev-1tb\CCM> docker run -d --name vol-test -v my-vol:/app nginx
docker: Error response from daemon: Invalid container name (-vol-test), only [a-zA-Z0-9][a-zA-Z0-9
_.-] are allowed

Run 'docker run --help' for more information
PS E:\dev-1tb\CCM> docker run -d --name vol-test -v my-vol:/app nginx
b954c5388030bd56cb5f1cca75a00807e729425ab60ad5dc4e21ad26bd6aba0b
PS E:\dev-1tb\CCM> docker exec -it vol-test sh -c "echo 'Data in volume' > /app/data.txt"

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker d
ebug vol-test
    Learn more at https://docs.docker.com/go/debug-cli/
PS E:\dev-1tb\CCM> docker stop vol-test
vol-test
PS E:\dev-1tb\CCM> docker rm vol-test
vol-test
PS E:\dev-1tb\CCM> docker run -it --rm -v my-vol:/app alpine cat /app/data.txt
Data in volume
PS E:\dev-1tb\CCM>

```

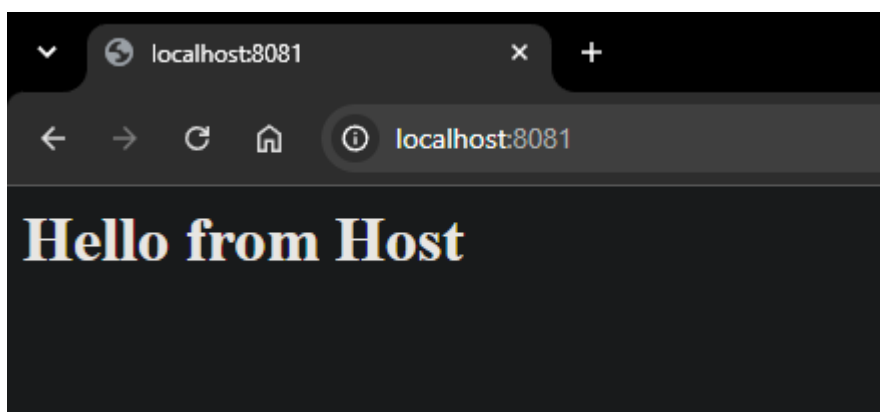
Part B — Bind Mount

```

mkdir html-data
echo "<h1>Hello from Host</h1>" > html-data/index.html
docker run -d --name bind-test -v "${PWD}/html-data:/usr/share/nginx/html" -p
8081:80 nginx

```

Screenshot



Written Answer (Lab 4.3)

Question: Which one is managed by Docker? Volume or Bind Mount?

Answer: Named Volumes are fully managed by Docker in a dedicated storage directory on the host system (`/var/lib/docker/volumes/`). Bind Mounts allow mounting any arbitrary directory or file from the host machine directly into a container and are managed manually by the user.

LAB 4.4 — Let Containers Talk to Each Other (Networks)

Goal: Connect containers using a custom user-defined network.

Steps

1. Create Network & Web Container:

```
docker network create my-net
docker run -d --name web-container --network my-net nginx
```

Screenshot

```
\CCM> docker network ls
NAME          DRIVER  SCOPE
bridge        bridge  local
host          host    local
my-net        bridge  local
none          null    local
```

```
PS E:\dev-1tb\CCM> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
15515a44e89b   nginx    "/docker-entrypoint...." 3 minutes ago  Up 3 minutes  80/tcp       web-con
tainer
```

2. Run Tester Container & Ping:

```
docker run -it --name tester --network my-net alpine
```

Inside container prompt:

```
ping web-container
```

(Press **Ctrl+C** to stop pinging, then type **exit**)

Screenshot

```
PS E:\dev-1tb\CCM> docker run -it --name tester --network my-net alpine
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
55afa1ecc21d: Pull complete
f5124fb579e2: Download complete
56dceff11b33: Download complete
Digest: sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db86916198a6eec434943f8b
Status: Downloaded newer image for alpine:latest
/ # ping web-container
PING web-container (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.086 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.089 ms
64 bytes from 172.18.0.2: seq=2 ttl=64 time=0.171 ms
64 bytes from 172.18.0.2: seq=3 ttl=64 time=0.078 ms
64 bytes from 172.18.0.2: seq=4 ttl=64 time=0.212 ms
64 bytes from 172.18.0.2: seq=5 ttl=64 time=0.063 ms
64 bytes from 172.18.0.2: seq=6 ttl=64 time=0.056 ms
64 bytes from 172.18.0.2: seq=7 ttl=64 time=0.210 ms
64 bytes from 172.18.0.2: seq=8 ttl=64 time=0.176 ms
64 bytes from 172.18.0.2: seq=9 ttl=64 time=0.173 ms
64 bytes from 172.18.0.2: seq=10 ttl=64 time=0.171 ms
64 bytes from 172.18.0.2: seq=11 ttl=64 time=0.169 ms
64 bytes from 172.18.0.2: seq=12 ttl=64 time=0.072 ms
64 bytes from 172.18.0.2: seq=13 ttl=64 time=0.138 ms
^C
--- web-container ping statistics ---
14 packets transmitted, 14 packets received, 0% packet loss
round-trip min/avg/max = 0.056/0.133/0.212 ms
/ #
```

3. Cleanup:

```
docker stop web-container
docker rm web-container tester
docker network rm my-net
```

```
PS E:\dev-1tb\CCM> docker stop web-container
web-container
PS E:\dev-1tb\CCM> docker rm web-container tester
web-container
Error response from daemon: cannot remove container "tester"
before removing or force remove
PS E:\dev-1tb\CCM> docker network rm my-net
Error response from daemon: error while removing network:
"tester" id:"83eaba8982e2")
exit status 1
```

Written Answer (Lab 4.4)

Question: Why use custom networks instead of the default bridge network?

Answer: Custom user-defined bridge networks enable automatic DNS container name resolution (containers can ping each other using container names), provide proper security isolation between containers, and support attaching/detaching containers on the fly without stopping them.

LAB 4.5 — Build Your Own Image (Dockerfile)

Goal: Build a custom Nginx image using a Dockerfile and deploy it.

Steps

1. Create Project Directory:

```
mkdir my-docker-app  
cd my-docker-app
```

2. Create `index.html`:

```
<h1>Welcome! My name is bryx</h1>
```

3. Create `Dockerfile`:

```
FROM nginx:alpine  
COPY index.html /usr/share/nginx/html/index.html
```

4. Build Image:

```
docker build -t my-custom-nginx .
```

5. Run Container:

```
docker run -d -p 8080:80 --name my-app my-custom-nginx
```

Open browser at: <http://localhost:8080>

Screenshot



Source Files

Dockerfile:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

index.html:

```
<h1>Welcome! My name is bryx</h1>
```

Written Answer (Lab 4.5)

Question: What does **COPY** do in the Dockerfile?

Answer: The **COPY** instruction copies local files or directories from the host build context into the filesystem of the container image at the specified destination path.